

Introduction to Matplotlib

Data Visualization in Python

What is Matplotlib?

Python's foundational plotting library

Used for creating static, interactive, and animated visualizations

Industry standard - most other libraries build on it

Getting Started

```
import matplotlib.pyplot as plt
```

Standard import convention - everyone uses `plt`

Your First Plot

```
days = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri']  
minutes = [30, 45, 20, 60, 40]  
  
plt.plot(days, minutes)  
plt.show()
```

Three steps:

1. Give it data (x and y lists)
2. Tell it what to plot (line, bar, scatter, etc)
3. Show it (displays the window)

The `plt.show()` is essential - nothing appears without it

The Pattern

Every matplotlib plot follows this pattern:

```
# 1. Data (can use plain Python lists)
x = [1, 2, 3, 4]
y = [10, 20, 15, 25]

# 2. Plot (connect the dots)
plt.plot(x, y)

# 3. Display (open the window)
plt.show()
```

Lists work fine - you don't need numpy for basic plots

Adding Labels

```
plt.plot(days, minutes)
plt.xlabel('Day of Week')
plt.ylabel('Practice Time (minutes)')
plt.title('My Weekly Practice Schedule')
plt.show()
```

Labels make your plots professional

Customizing Appearance

```
plt.plot(days, minutes,  
         color='green',  
         marker='o',  
         linewidth=2)  
plt.grid(True)  
plt.show()
```

`color` - any color name or hex code like '#FF5733'

`marker` - 'o' circle, 's' square, '^' triangle, etc

`linewidth` - thickness of line (default is 1)

`grid(True)` adds gridlines for easier reading

More Style Options

```
plt.plot(x, y,  
         linestyle='--',    # dashed line  
         alpha=0.7,        # transparency (0-1)  
         label='My Data')  # for legend  
  
plt.legend()  
plt.show()
```

`linestyle` - '-' solid, '--' dashed, ':' dotted, '-.' dash-dot

`alpha` - transparency: 1 is opaque, 0 is invisible

`label` - name for the legend

Bar Charts

```
tunes = ['Butterfly', 'Kesh', 'Silver Spear']  
plays = [5, 8, 3]  
  
plt.bar(tunes, plays)  
plt.xlabel('Tune Name')  
plt.ylabel('Times Played')  
plt.show()
```

Use bar charts for categorical data (categories, not continuous numbers)

Line plots for trends over time

Bar plots for comparing distinct categories

Rotating Long Labels

```
plt.bar(tunes, plays)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

`rotation=45` tilts labels so they don't overlap

`ha='right'` aligns them properly (ha = horizontal alignment)

`tight_layout()` prevents labels getting cut off at edges

Multiple Lines

```
plt.plot(hours, alice, label='Alice')  
plt.plot(hours, bob, label='Bob')  
plt.legend()  
plt.show()
```

Add `label` parameter and call `plt.legend()`

Subplots

```
fig, (ax1, ax2) = plt.subplots(1, 2)

ax1.plot(x, y1)
ax1.set_title('First Plot')

ax2.plot(x, y2)
ax2.set_title('Second Plot')

plt.show()
```

Create multiple plots side by side

`subplots(rows, columns)` - here 1 row, 2 columns

Returns unpacked into ax1 and ax2

Subplots Explained

```
plt.subplots(rows, columns)
```

Returns: figure object and axes object(s)

Use `ax.plot()` instead of `plt.plot()`

Use `ax.set_xlabel()` instead of `plt.xlabel()`

This is the "object-oriented" interface - you're calling methods on objects

Subplots - Vertical Stack

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 8))
```

2 rows, 1 column - stacked vertically

`figsize=(width, height)` in inches

Default is (6.4, 4.8) which is often too small

Subplots - Grid Layout

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

axes[0, 0].plot(data1) # top-left
axes[0, 1].plot(data2) # top-right
axes[1, 0].plot(data3) # bottom-left
axes[1, 1].plot(data4) # bottom-right

plt.tight_layout()
plt.show()
```

For grids, `axes` is a 2D array

Access with `axes[row, col]`

Saving Plots

```
plt.plot(x, y)  
plt.savefig('my_plot.png')
```

Instead of `show()`, use `savefig()`

Supports: .png, .jpg, .pdf, .svg

File format determined by extension

Saving with Quality Options

```
plt.plot(x, y)
plt.savefig('my_plot.png',
            dpi=300,
            bbox_inches='tight')
```

`dpi=300` for high resolution (default is 100)

`bbox_inches='tight'` prevents cutting off labels

Good for reports and presentations

Common Plot Types

Line plot: `plt.plot(x, y)`

Bar chart: `plt.bar(x, y)`

Scatter plot: `plt.scatter(x, y)`

Histogram: `plt.hist(data)`

Example: Line Plot

```
x = [0, 1, 2, 3, 4]
y = [0, 1, 4, 9, 16]

plt.plot(x, y, marker='o')
plt.xlabel('x')
plt.ylabel('y = x2')
plt.title('Quadratic Function')
plt.grid(True)
plt.show()
```

Example: Scatter Plot

```
x = [1, 2, 3, 4, 5, 6, 7, 8]
y = [2, 4, 3, 6, 5, 8, 7, 9]

plt.scatter(x, y, s=100, alpha=0.6)
plt.xlabel('X values')
plt.ylabel('Y values')
plt.title('Scatter Plot Example')
plt.show()
```

Example: Histogram

```
scores = [67, 72, 88, 91, 65, 77, 82, 95, 71]

plt.hist(scores, bins=5, edgecolor='black')
plt.xlabel('Score Range')
plt.ylabel('Frequency')
plt.title('Exam Scores')
plt.show()
```

Key Takeaways

Import: `import matplotlib.pyplot as plt`

Pattern: data → plot → show

Always add labels: xlabel, ylabel, title, legend

Two interfaces: pyplot (simple) and OO (complex layouts)

Tkinter: use Figure(), canvas.draw(), never plt.show()

Save with: `plt.savefig('file.png', dpi=300)`

Why Two Interfaces?

Matplotlib has two ways to create plots

This is historical - pyplot came first to mimic MATLAB

Later, the object-oriented interface was added for better control

You'll see both in documentation and examples online

The Pyplot Interface

```
plt.plot(x, y)  
plt.xlabel('X')  
plt.title('Title')  
plt.show()
```

Convenient for quick plots

Works with a hidden "current" figure and axes

Good for exploration and simple scripts

The Object-Oriented Interface

```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_xlabel('X')
ax.set_title('Title')
plt.show()
```

Explicit control - you own the figure and axes objects

Required for multiple plots and complex layouts

Better for building applications

When to Use Which?

Use pyplot (`plt.plot`) when:

- Quick data exploration
- Single simple plot
- Interactive notebook work

Use OO (`fig, ax`) when:

- Multiple subplots
- Building a GUI application
- Need precise control over plot elements

Understanding the Objects

```
fig, ax = plt.subplots()
```

`fig` is the Figure - the entire window/canvas

`ax` is the Axes - the actual plot area (where data appears)

One figure can contain multiple axes

Multiple Subplots Example

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))

ax1.plot(x, y1, color='red')
ax1.set_title('First Plot')

ax2.bar(categories, values, color='blue')
ax2.set_title('Second Plot')

plt.tight_layout()
plt.show()
```

This is why OO interface exists - clean control of multiple plots

Embedding in Tkinter

You can embed matplotlib plots in your tkinter GUIs

Key: use `FigureCanvasTkAgg` to bridge matplotlib and tkinter

Don't use `plt.show()` - use `canvas.draw()` instead

Basic Tkinter Integration

```
import tkinter as tk
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

root = tk.Tk()

fig = Figure(figsize=(6, 4))
ax = fig.add_subplot(111)
ax.plot([1, 2, 3], [1, 4, 9])

canvas = FigureCanvasTkAgg(fig, master=root)
canvas.draw()
canvas.get_tk_widget().pack()

root.mainloop()
```

Tkinter Integration Explained

`Figure()` creates the plot (not `plt.figure()`)

`fig.add_subplot(111)` creates the axes

`FigureCanvasTkAgg(fig, master=root)` embeds in tkinter

`canvas.get_tk_widget()` returns a tkinter widget you can pack/grid

`canvas.draw()` refreshes the display

Interactive Tkinter Plot

```
class PlotApp:
    def __init__(self, root):
        self.fig = Figure()
        self.ax = self.fig.add_subplot(111)

        self.canvas = FigureCanvasTkAgg(self.fig, root)
        self.canvas.get_tk_widget().pack()

        btn = tk.Button(root, text="Update",
                        command=self.update_plot)
        btn.pack()

    def update_plot(self):
        self.ax.clear()
        # plot new data
        self.canvas.draw()
```

Updating Pattern in Tkinter

```
def update_plot(self):  
    # 1. Clear the old plot  
    self.ax.clear()  
  
    # 2. Plot new data  
    self.ax.plot(new_x, new_y)  
    self.ax.set_title('Updated Plot')  
  
    # 3. Redraw the canvas  
    self.canvas.draw()
```

Always follow this pattern: clear, plot, draw

`self.ax.clear()` wipes the previous plot

`self.canvas.draw()` refreshes the display

Complete Tkinter Example

```
import tkinter as tk
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

class App:
    def __init__(self, root):
        self.multiplier = tk.IntVar(value=2)

        tk.Scale(root, from_=1, to=10,
                 variable=self.multiplier,
                 command=lambda x: self.update()).pack()

        self.fig = Figure()
        self.ax = self.fig.add_subplot(111)
        self.canvas = FigureCanvasTkAgg(self.fig, root)
        self.canvas.get_tk_widget().pack()
        self.update()

    def update(self):
        self.ax.clear()
        x = range(10)
        y = [self.multiplier.get() * i for i in x]
        self.ax.plot(x, y)
        self.canvas.draw()

root = tk.Tk()
App(root)
root.mainloop()
```

Key Differences: Standalone vs Tkinter

Standalone:

- Use `plt.figure()` or `plt.subplots()`
- End with `plt.show()`

Tkinter:

- Use `Figure()` (import from `matplotlib.figure`)
- Use `canvas.draw()` to display/update
- Never call `plt.show()` - it creates a separate window

Practice Exercise

Create a visualization about yourself:

- Use bar chart OR line plot
- Add proper labels and title
- Save it as a .png file

Bonus challenge: Embed it in a tkinter window with an update button

Ideas: hobbies, study time, favorite things, daily screen time

Next Steps

Practice with your own data

Experiment with colors and styles

Next lesson: Adding numpy for more power

Questions?

Remember: `plt.show()` to display

Documentation: matplotlib.org